香山昆明湖 V2

自定义向量点积指令

第三阶段：协同仿真与性能评估

设计、验证与性能评估

三阶段工程技术报告

2026 年 11 月 8 日修订

目录

第三阶段：协同仿真与性能评估 **1**

3.1 从功能验证到工程评价 **1**

3.2 NEMU 参考语义与处理器接入 **2**

3.3 差分检验与错误检出 **2**

3.4 全核波形中的点积事务 **3**

3.4.1 源数据、结果与身份 3

3.4.2 从有效区间到握手边沿 3

3.4.3 输入与接受 4

3.4.4 结果与响应 5

3.4.5 提交与架构目的 5

3.5 等价 RVV 对照与测量方法 **6**

3.5.1 完整窗口与配对统计 6

3.6 硬件微基准结果 **7**

3.7 软件调用与 C8 优化 **7**

3.8 完整前向计算的组织与验证 **8**

3.8.1 从点积内核到模型计算链 8

3.8.2 工作量与数据布局 9

3.8.3 完整计算链的验证结果 10

3.9 综合面积、时序与设计成本 **10**

3.9.1 SP1 一拍运算方案的对比 11

3.9.2 不同工艺库下的参考综合 13

3.10 设计取舍与应用方向 **14**

3.11 补充：三种标准 RVV 实现的完整前向观测 **15**

3.12 补充：C8 软件包的接入与镜像核对 **15**

3.12.1 固定平台与数据布局 16

3.12.2 编译期选择与调用 16

3.12.3 从函数对象到可执行镜像 16

3.13 评估对象与证据索引 **17**

3.13.1 复现与图像来源 18

第三阶段：协同仿真与性能评估

3.1 从功能验证到工程评价

自定义指令需要在程序中产生正确结果，也需要与处理器的异常、状态和调度机制配合。本阶段使用独立 NEMU 参考模型和全核 DiffTest 核对架构行为，再以等价 RVV 程序比较完整测量窗口的周期。另外给出软件接入方式，以及综合面积和静态时序结果，用于评估这项设计的收益与成本。主要结果是：RTL 与 NEMU 参考模型在全部功能用例上协同仿真一致；相对等价的 RVV 程序，独立批量和依赖链场景分别加速 3.07 倍和 2.93 倍，单次点积加速 1.20 倍；独立批量的收益主要来自两槽流水，它比上一版单笔在途设计少用 39% 的周期；8×32 GEMV 函数调用快 1.095 倍；VDot 功能单元的面积约占所在执行单元的 6.0%，在 4 ns 约束下所有路径都满足 setup 要求。

本阶段的评估对象是最终提交的稳定版，即八项点积、两槽流水的实现（下文称“受测版本”），依次给出微基准、C8 软件调用和完整前向计算的结果。VDot 功能单元下文简称 FU；它所在的向量执行单元 VFEX4 与整数除法、浮点除法/开方单元共享派发和写回，下文称为父级。第3.9节还在相同综合条件下比较了 SP1 方案（把点积运算从两拍缩短为一拍），并附一组双工艺库参考数据。比较之后，最终提交仍采用稳定版，SP1 作为优化探索保留。每组实验都注明了所用版本、输入和测量范围。

图3-1给出本阶段的协同仿真框架。同一 AM 镜像与共同启动 flash 分别加载到香山 DUT 和固定 NEMU 参考端；DUT 按架构提交推进参考模型，两端状态送入 DiffTest，比较完整 128 位向量结果、约定的向量 CSR 和异常状态。运行配置冻结为单核、VLEN128 和 128MB 内存，源码、二进制、配置、原始日志和真实退出码一并归档。两端的点积计算方式不同，参考端的具体处理见下节。

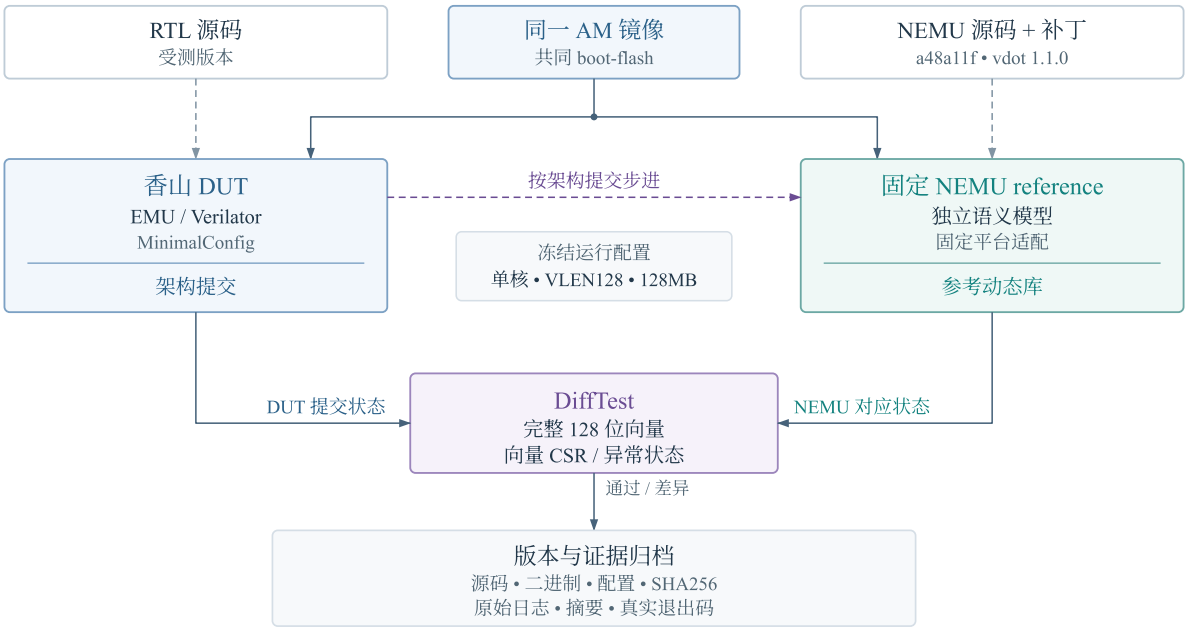


图 3-1 RTL 与独立参考模型的协同仿真

3.2 NEMU 参考语义与处理器接入

RTL 通过并行乘法、归约网络和流水状态执行点积；NEMU 参考模型则从指令字、源寄存器和向量配置出发，顺序计算八项有符号乘积之和。两者的内部实现方式不同，因此可以相互核对。

参考端首先匹配 custom-0 编码并检查合法配置，随后快照读取两个源寄存器，计算点积并构造完整 128 位目的值：低 32 位为整数结果，高 96 位为零。快照读取使源、目的寄存器重合时仍使用本条指令执行前的数据。合法运算保留约定的向量状态；非法配置进入异常处理。

受测版本使用基于 NEMU 固定版本的 vdot 补丁及配套平台适配。适配使 NEMU 的 xTVAL 行为与该香山版本一致，异常码、陷阱 PC、xTVAL 和目的寄存器状态都在比较范围内；特权相关测试在 M 模式下完成。正常参考库与用于负控的故障注入库分别构建，各自记录来源和 SHA-256。

图3-2展示参考端的处理流程。参考端先精确匹配 custom-0 编码，未匹配的指令交给其他译码；再检查向量状态是否合法，即 VS 已启用、vtype 合法、vl=8 且 vstart=0，非法时产生 cause=2 的非法指令异常且不写 vd；合法时先快照两源的低 64 位，执行 8 项有符号点积，再完整写回 128 位结果，并在成功后将 VS 置脏。

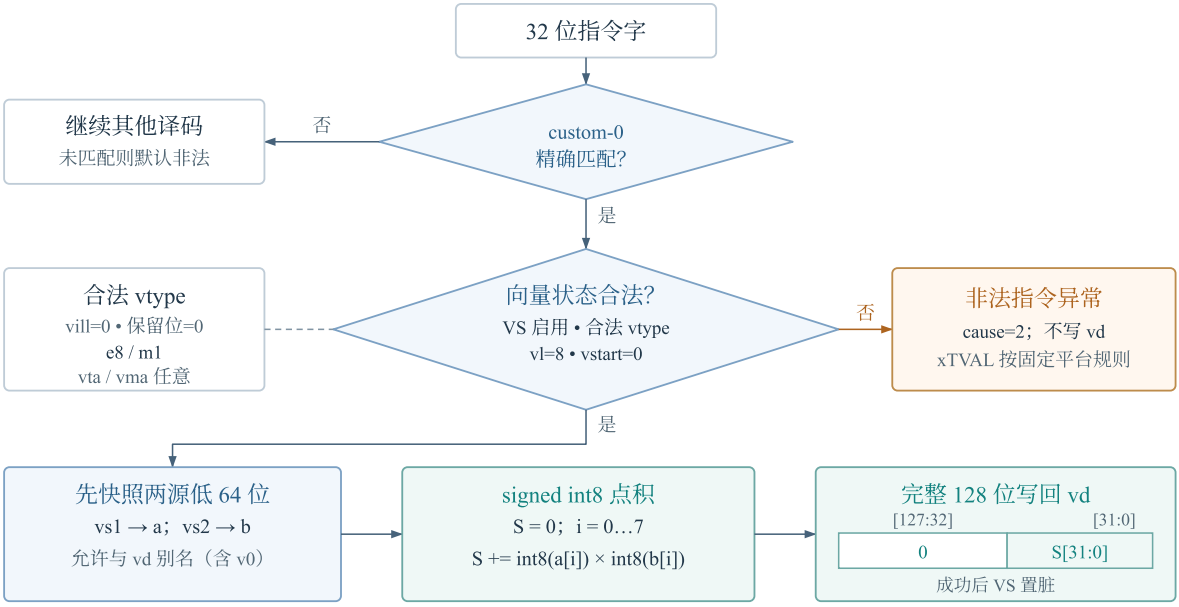


图 3-2 参考模型的译码、状态检查与完整写回

3.3 差分检验与错误检出

基础功能矩阵覆盖正数、负数、边界值、寄存器别名和非法向量配置。在受测版本上，全核矩阵包含 117 个不同用例，按矩阵分片共执行 124 次；数值正确性由算术 RTL 的 10022 项测试覆盖，握手控制由真实父级测试覆盖。最终验收时汇总了 13 项已有的全核记录和 1 项新增的控制诊断运行，并重新解析了它们的原始输出。

为了检验完整 128 位比较是否有效，负控在参考端的 vdot 结果中翻转 bit100。差分器检测到目的寄存器高半区不一致并按预期终止；恢复正常参考模型后，同一程序完成 15 项检查并到达 GOOD TRAP。图3-3给出了这两个互补结果：前者检验错误检出能力，后者检验正常执行路径。负控日志中可见 `v10_high different`、`ABORT` 和退出码 1，差异出现在 PC `0x80001bdc`；日

志里的 `right` 和 `wrong` 只是比较器两侧的名称，本例的错误注入在参考端。正常日志的退出码为 0。两段日志分别来自干净环境复现中的 `negative-vdot` 与 `functional-0` 运行。

（a）参考端翻转 bit100：差分器检出高位差异

```
PS C:\Users\33532> Start-Process 'D:\sharex\ShareX\pages\2026-11\Notepad_kUWl09ct1X.png'
PS C:\Users\33532> wsl -d Ubuntu-22.04 -- tail -n 7 /home/riscv/riscv-lab/software-import-20261026/clean-02/fullcore-run-01/runs/negative-vdot/run.log
privilegeMode: 3
v10_high different at pc = 0x0080001bdc, right= 0x0000001000000000, wrong = 0x0000000000000000
Core 0: ABORT at pc = 0x800016b0
Core-0 instrCnt = 238849, cycleCnt = 311097, IPC = 0.767764
Seed=0 Guest cycle spent: 311100 (this will be different from cycleCnt if emu loads a snapshot)
Host time spent: 740557ms
PS C:\Users\33532> wsl -d Ubuntu-22.04 -- cat /home/riscv/riscv-lab/software-import-20261026/clean-02/fullcore-run-01/runs/negative-vdot/exit-code.txt
1
PS C:\Users\33532>
```

（b）恢复正常参考模型：15 项检查通过

```
PS C:\Users\33532> wsl -d Ubuntu-22.04 -- tail -n 6 /home/riscv/riscv-lab/software-import-20261026/clean-02/fullcore-run-01/runs/functional-0/run.log
VDOT_SUMMARY mode=full contract=vdot-s8x8-r32-v0.1 planned=15 executed=15 passed=15 failed=0 unreachable=0
Core 0: HIT GOOD TRAP at pc = 0x8000236a
Core-0 instrCnt = 580586, cycleCnt = 737849, IPC = 0.786863
Seed=0 Guest cycle spent: 737853 (this will be different from cycleCnt if emu loads a snapshot)
Host time spent: 1663260ms
PS C:\Users\33532> wsl -d Ubuntu-22.04 -- cat /home/riscv/riscv-lab/software-import-20261026/clean-02/fullcore-run-01/runs/functional-0/exit-code.txt
0
```

图 3-3　高位错误检出与正常参考模型恢复的原始日志

3.4　全核波形中的点积事务

为观察点积在全核中的输入、响应和提交，2026 年 11 月 5 日使用受测版本在干净环境复现中生成的 RTL，单独构建了可输出 FST 波形的 EMU。该模型运行原有的全核功能测试程序 functional-2，加载正常 NEMU 参考库并启用 DiffTest，最终到达 GOOD TRAP、退出码 0。以下波形将同一笔正数点积从执行输入关联到架构提交。

3.4.1　源数据、结果与身份

两个源寄存器均为 0xa5a5a5a5a5a5a5a550807060504030201，低八个有符号字节依次为 1 至 8。预期结果为

$$1^2 + 2^2 + \cdots + 8^2 = 204.$$

实际 128 位响应为 0x000000000000000000000000000000cc，符合低 32 位保存结果、高 96 位清零的布局。源的高 64 位填入 A5 作为哨兵，用来确认硬件只取了低 8 个元素。表3-1列出用于关联这笔事务的 ROB、物理目的、提交观察信号、PC 和机器字。

表 3-1　正数点积事务的关联字段

关联字段	观测值
ROB 与物理目的	ROB 编号 39（十六进制 27，环回标志为 0）；物理目的 103（十六进制 67）
提交观察	`io_valid=1`、`io_vecwen=1`；架构目的 v10
PC 与机器字	PC=`0x80001b8c`；机器字 `0x0294050b`

3.4.2　从有效区间到握手边沿

波形中的 valid 表示一段有效区间，事务接受则发生在 valid 与 ready 同时有效的时钟上升沿。表3-2把两者分开列出，便于与日志中的周期坐标对照。

表 3-2 点积事务关键事件的波形坐标

事件	显示坐标（ps）	观察依据
输入有效（A）	678694	输入 valid、ready 均为 1，源值及身份有效
输入接受边沿	678696	边沿前握手有效，边沿后 P 槽有效
输出有效（B）	678698	R 槽有效，输出 valid、ready 均为 1，结果 204
输出握手边沿	678700	边沿前输出有效，边沿后 R 槽清空
DiffTest 提交观察（C）	678710	提交有效，PC、编码、ROB 与目的相符

波形每个全局周期占两个坐标单位，观察器日志则直接按周期计数，两者的起点不同。同一事务在观察器中的接受、响应和内部 ROB 提交周期分别为 339296、339298 和 339301，对应波形坐标 678696、678700 和 678706，三者换算后都相差固定的 52 个周期，相对间隔完全一致。接受到响应相隔 2 个周期，与第二阶段的两槽时序一致；内部提交之后，DiffTest 观察口又经过 2 个周期（坐标 678710）才显示这次提交。分析时延时只使用周期差，不比较两套坐标的绝对值。

后面三幅截图中，上方为全局时钟，下方为点积单元的门控时钟，单元空闲时该时钟可以停止。截图中的 ps 只是波形工具的显示单位。

图3-4至图3-6三幅 GTKWave 截图分别展示输入、响应和提交。截图保留信号值，仅裁去层级栏和外部空白；完整原图、事务采样和运行回执保存在团队仓库的对应实验归档中。

3.4.3 输入与接受

如图3-4所示，游标 678694 处输入 valid 和 ready 均为 1，两个源的低八字节为 1 至 8，ROB 编号和物理目的按十六进制显示为 27 和 67（即 39 和 103）；678696 上升沿接收请求，此后 P 槽有效。

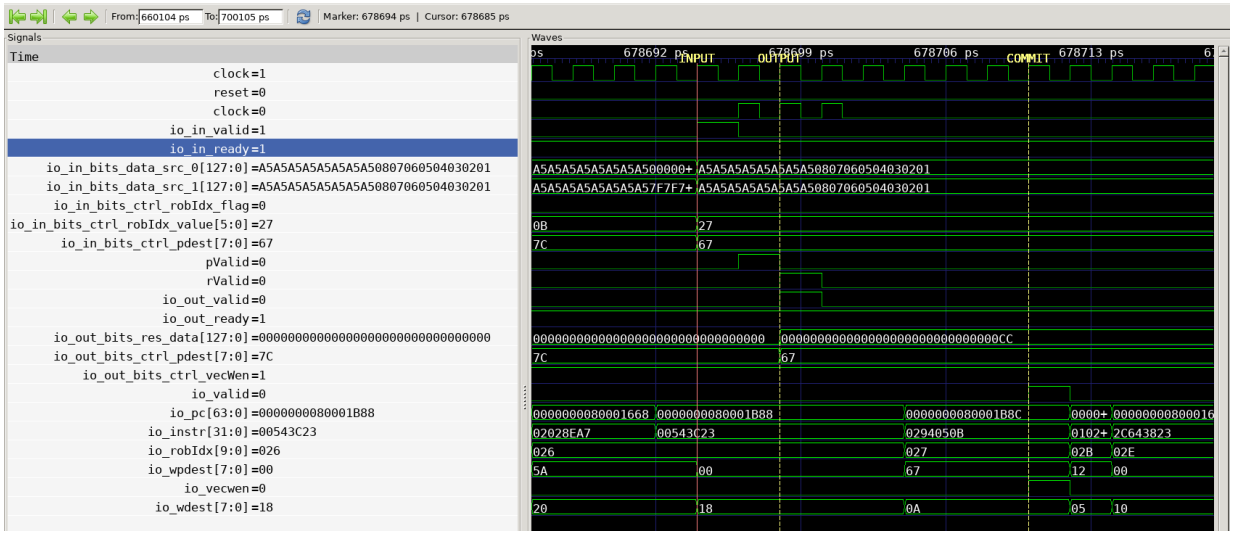


图 3-4 正数点积的输入与接受边沿

3.4.4 结果与响应

图3-5中，游标 678698 处 R 槽和输出 valid、ready 有效，结果为 204 且高 96 位为零；678700 边沿完成握手并释放 R 槽，此时提交观察口的 io_valid 仍为 0。

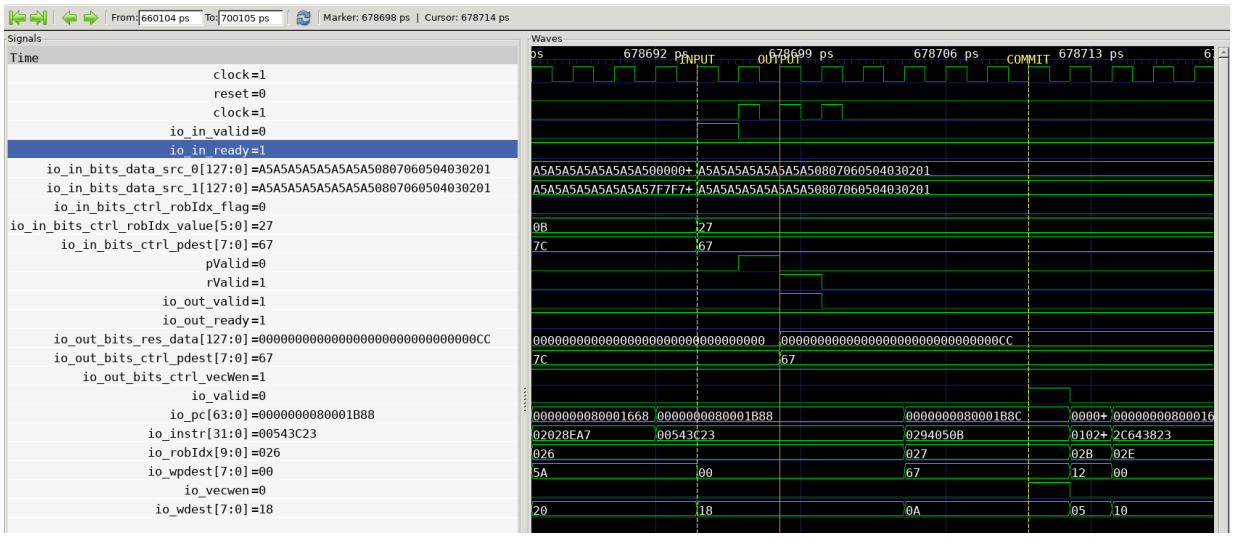


图 3-5 完整 128 位结果与响应握手

3.4.5 提交与架构目的

图3-6中，游标 678710 处提交有效，PC 为 80001B8C，机器字为 0294050B，ROB 显示为 027（环回标志 0、编号 27），物理目的为 67，架构目的为 0A（v10），向量写使能有效，均对应同一事务；观察口相对内部提交的两拍延迟已在前文说明。

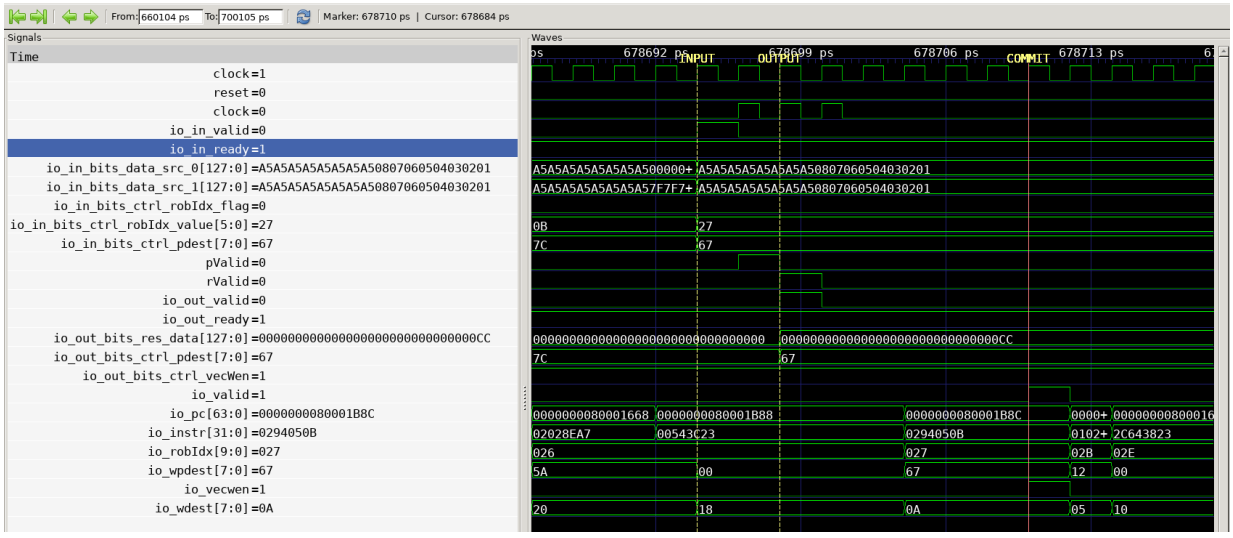


图 3-6 DiffTest 提交观察与架构目的

3.5 等价 RVV 对照与测量方法

8 项点积微基准中的标准 RVV 对照，通过宽乘和宽归约计算与 VDot 相同的有符号点积。单次已知配置序列先以 `vwmul.vv` 得到 16 位乘积，再切换到 `e16/m1/VL8`，清零整个目的寄存器并执行 `vwredsum.vs`，最后恢复入口配置。高 96 位在归约过程中保留为零，低 32 位保存完整整数和。

该微基准按使用场景组织：已知配置的单次点积为 5 条主体指令，通用入口增加 `vl/vtype` 保存，共 7 条；四个独立点积共享配置切换，主体合计 14 条。批处理时多个点积共用配置切换，标准 RVV 也能充分利用独立计算；比较时双方实际执行的清零、数据搬运和状态管理都计入周期。

3.5.1 完整窗口与配对统计

测量采用同一输入、循环次数、镜像配置和边界规则。每个微基准场景使用三组数据，每组统计四个正式窗口；首次、预热和独立重放分别保存。计时保留实际执行的配置切换、循环与边界开销，统计单位为 DUT 全局周期。

香山是乱序处理器，计数器的读取时刻需要与指令执行位置对应。项目在受测版本上组合使用 fence 与 cycle、instret 读数，并通过观察器关联四条 CSR 指令的 PC、机器字、ROB 身份、采样和退休事件。cycle 差与全核采样 tick 差相符，instret 差与退休记录及 ELF 展开计数相符；计数器读数均按整数记录。

图3-7按观察器事件绘出这一对应关系。四条 CSR 读取指令 I0、C0、C1、I1 分别标出进入 ROB、CSR 采样和退休三个时刻，横轴为相对 START 的全核 tick。cycle 采样差为 $136-65=71$，与 tick 差相等；instret 差为 12，按退休 count 求和，计数区间包含 I0 的退休、不包含 I1 的退休，12 项由 3 条 CSR、4 条 fence 和 5 条主体指令组成，主体中含 1 条 vdot。

加速比与周期降低率分别定义为

$$S=\frac{C_{\mathrm{RVV}}}{C_{\mathrm{VDot}}},\qquad R=\frac{C_{\mathrm{old}}-C_{\mathrm{new}}}{C_{\mathrm{old}}}.$$

两者的分母不同，因此表中同时给出原始周期。完整窗口还包含调度、访存、循环和边界开销，窗口周期与第二阶段所说的 FU 两拍响应是两种不同的计量。

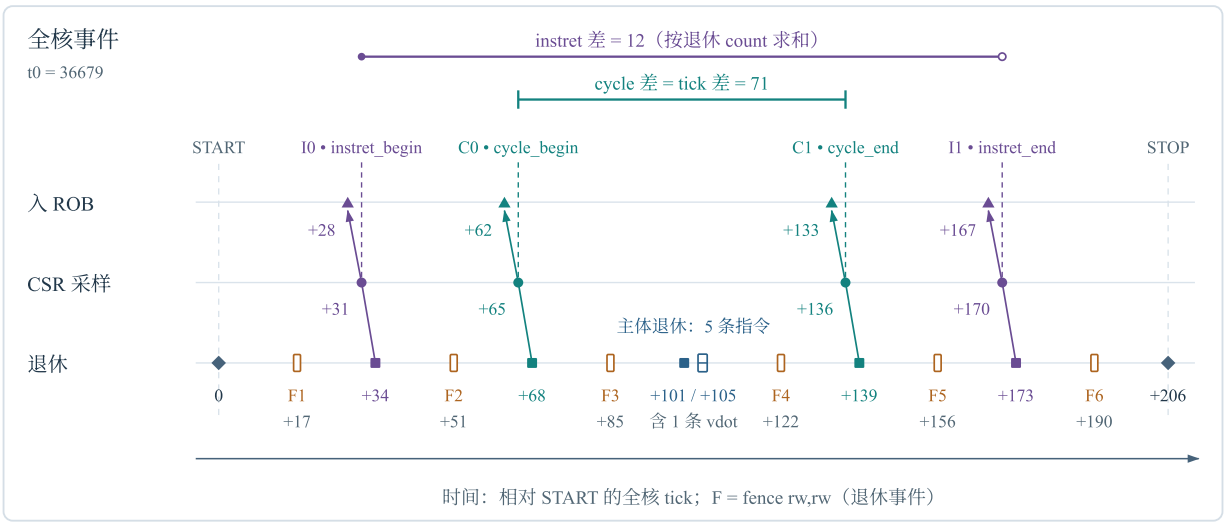


图 3-7 CSR 采样、退休与计时窗口的对应关系

3.6 硬件微基准结果

四类微基准分别观察单次点积、连续数据依赖、独立任务吞吐，以及加载、计算与存储共同作用下的表现。表3-3使用同镜像中的 VDot 与等价 RVV 路径；每个场景、每种实现使用三个数据集，每个数据集取四个正式窗口，共十二窗。

表 3-3 受测版本的微基准结果（周期均值）

场景	VDot	RVV	加速比
单次点积	71	85.25	1.2007×
依赖链（256 次）	1857	5440.5	2.9297×
四路独立批量（128 次）	221	678	3.0679×
加载、计算、存储流（64 次）	1127.5	1133	1.0049×

加速比为同一场景 RVV 与 VDot 的平均周期之比。独立批量由 32 轮四路点积组成，能够持续向流水线提供相互独立的工作；依赖链则要求每次点积使用前次结果。对比这两个场景可以看出，任务之间是否存在数据依赖对收益影响很大。加入加载、存储与循环控制后，stream64 两条路径的周期接近，说明只加快点积本身，并不能明显缩短整个循环的时间。

独立批量的高加速比主要来自第二阶段的两槽设计：与上一版单笔在途设计相比，两槽设计在同一镜像中把独立批量从 362.5 周期降到 221 周期，减少 39.0%。DIV 取消修复不影响周期，加入前后同一输入的 96 对正式窗口周期完全相同。

图3-8以同场景 RVV 周期为 100% 进行归一化。VDot 在依赖链和独立批量场景中分别只需 RVV 周期的 34.13% 和 32.60%，单次点积为 83.28%，访存流为 99.51%，与 RVV 基本持平；这些比例与表3-3的加速比相对应。

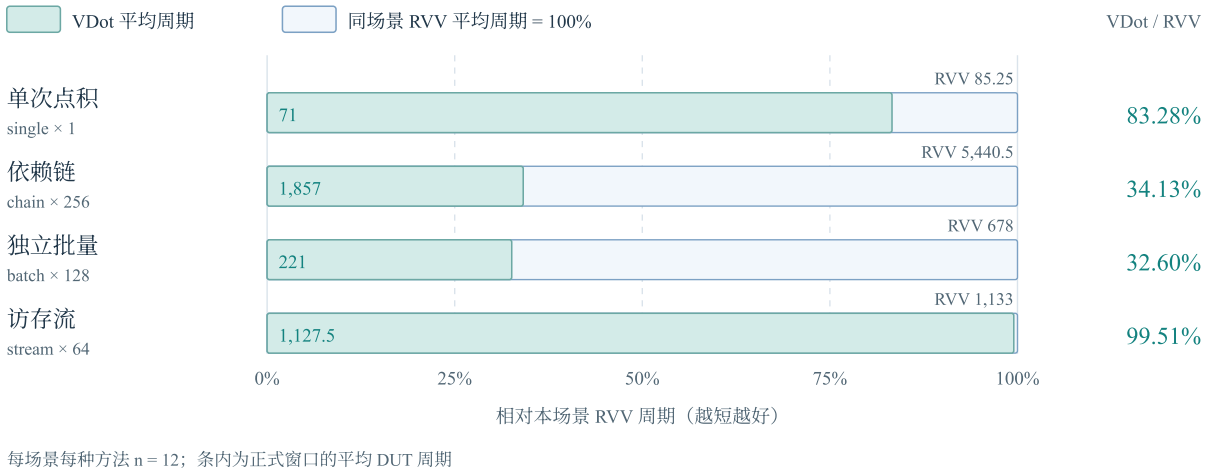


图 3-8 四类微基准周期与 RVV 归一化结果

3.7 软件调用与 C8 优化

前述微基准考察 8 项点积的指令序列，本节转向 8 行、每行 32 项 GEMV 的完整函数调用。硬件点积单元要在矩阵运算中发挥作用，还需要减少相邻计算之间的配置和数据搬运。首段初始化

版本（firstdot）用首段点积直接初始化累加状态，随后接续其余分段；C8 版本在复用输入向量的 8 行、每行 32 项 GEMV 中，进一步消除重复的向量配置。两者分别减少累加初始化与配置管理的开销。

C8 提供普通 RV64 C 接口。调用者将行优先矩阵 $A[8][32]$ 和输入向量 $x[32]$ 连续存放，形成 288 字节输入；返回值写入 8 个 16 字节结果槽，每槽低 32 位保存 INT32 点积，高 96 位清零。槽布局与指令的 128 位结果格式相同，结果可以直接用于功能检查和后续处理。

VDot 路径将每行分为四个 8 项点积段；RVV 路径则处理两个 16 项半组，以 `e8/m1/VL16` 宽乘和 `e16/m2/VL16` 宽归约接续得到完整 32 项组和。两种实现共享输入布局与完整结果格式，各自采用适合的分组和配置复用方式。三种实现的周期均值见表3-4。

表 3-4 8×32 整数 GEMV 调用的周期均值

8×32 整数 GEMV 调用	周期均值	对照意义
首段初始化版本（VDot）	282.25	上一版软件
C8 配置复用版本（VDot）	273.00	本节选用实现
较快的 RVV 实现	299.00	标准指令对照

在受测版本上的这组 8×32 配对测量中，输入已打包为 INT8，输出为 8 个完整结果槽。C8 比首段初始化版本少用 3.3% 的周期，比较快的 RVV 实现快 1.095 倍。统计采用三个数据集、每组四个正式窗口，包含完整调用开销。这些结果说明，整数点积之外的软件组织仍然影响最终收益。[1]

接入时，`xs_c8_gemv_8x32` 在编译期选择对应函数，显式 RVV 入口提供接口相同的对照版本。封装进软件包后，最终程序与原测试程序的 31,440 字节 BIN 逐字节一致，装载段、48 项调用表和 12 组输入也相同。[2]由此可以确认，经过接口封装后实际执行的代码与测试时一致，新应用可以直接调用这些函数。

图3-9概括了 C8 的软件入口和最终镜像核对。上半部分为普通 C 调用：288 字节的连续输入交给 `xs_c8_gemv_8x32` 或显式 RVV 入口处理，编译期开关决定前者连接 VDot 还是 RVV，输出为 8 个 16 字节结果槽，每槽低 32 位为结果、高 96 位清零；下半部分是三方最终镜像的一致性核对，说明封装后的程序与测试时完全相同。接口约定和镜像核对的细节见第3.12节。

3.8 完整前向计算的组织与验证

本节验证 VDot 点积内核能否正确接入一个完整的模型计算：把量化矩阵向量乘内核接入一个小型 Transformer 的完整前向计算，分别使用标准 RVV 内核和 VDot 内核运行，并检查整个模型的计算结果。本节的重点是正确性，周期为单次观测。

3.8.1 从点积内核到模型计算链

应用验证将量化矩阵向量乘接入一个六层 Transformer 的完整 forward。固定合成模型的模型维度为 288，前馈中间维度为 768，注意力头数为 6，词表大小为 32,000；测试在规定初始化后，

[1]C8 接口和结果见C8 软件包验收结果及其固定输入清单。

[2]三方最终镜像比较见C8 最终镜像等价结果。

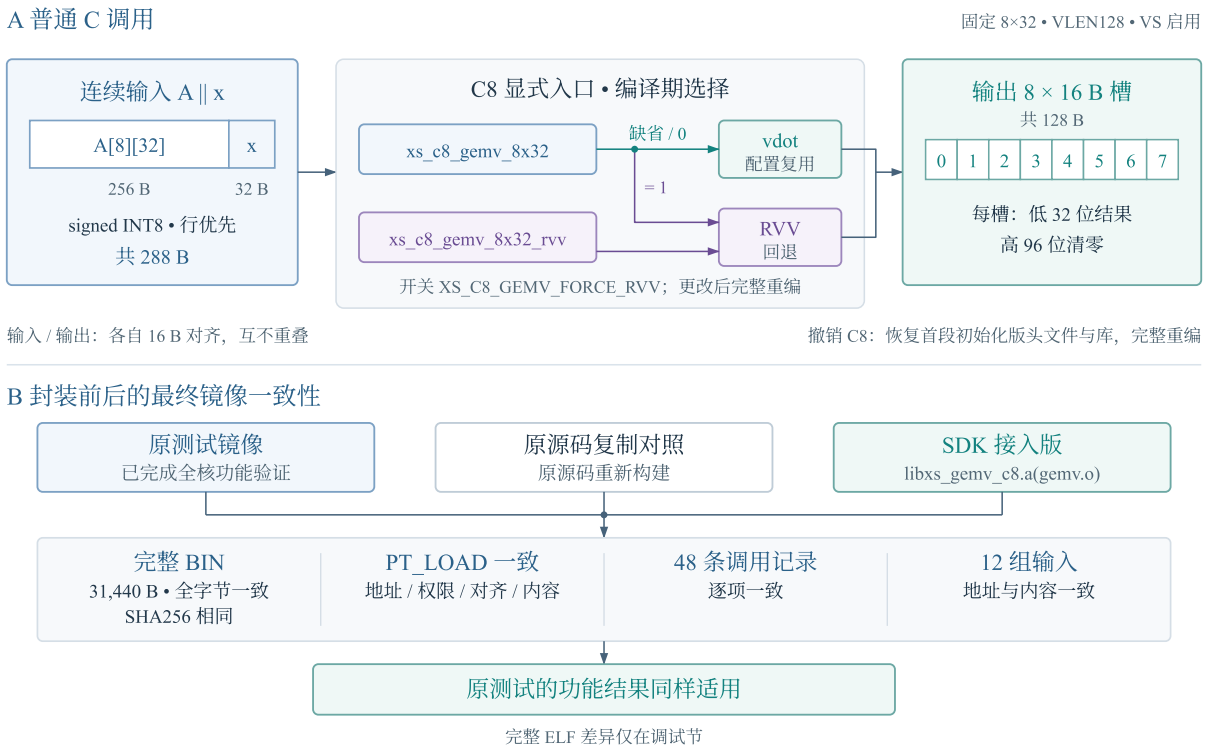


图 3-9 C8 软件入口、结果布局与最终镜像等价核对

执行 token=1、position=0 的一次前向计算。权重与尺度预先完成打包，该准备成本不计入本次窗口。计数窗口覆盖一次完整 forward 调用，包含激活量化；计算依次经过嵌入读取、各层归一化、Q/K/V 投影、位置编码、注意力、输出投影及前馈网络，最终生成 32,000 项分类器输出。

本节的量化 GEMV 内核按每 32 项一组计算，直接生成 FP32 行输出。每组先求精确的 INT32 点积，再转换为 FP32，依次乘以权重尺度和激活尺度，最后按组号顺序累加到输出。不同内核共用权重、尺度数组及这一浮点计算顺序。Q、K、V 投影复用同一份量化激活，前馈网络的两个输入投影也共享量化结果。

3.8.2 工作量与数据布局

表3-5列出一次 forward 中的 43 个量化 GEMV。这里以一项乘加记一个 MAC，统计矩阵的有效维度；注意力、归一化、激活函数等运算不计入该表，但包含在完整 forward 中。

表 3-5 固定模型一次 forward 的量化 GEMV 工作量

矩阵类别	数量	每个矩阵的形状	逻辑 MAC	占比
Q/K/V/O 投影	24	288×288	1,990,656	13.11%
前馈 W1/W3	12	768×288	2,654,208	17.48%
前馈 W2	6	288×768	1,327,104	8.74%
分类器	1	$32,000 \times 288$	9,216,000	60.68%
合计	43	—	15,187,968	100.00%

分类器占逻辑 MAC 的 60.68%，因此大矩阵路径最值得优化。这个比例描述乘加工作量；其

实际耗时还取决于访存、尺度处理、循环控制及处理器执行资源。

现有布局将权重按八行和量化组打包，尺度沿每个输出行连续存放。输入维度为 288 的 37 个矩阵按每次 288 个输出行调用点积内核，共 172 次：Q/K/V/O 各 1 次，W1/W3 各 3 次，分类器 112 次。输入维度为 768 的 6 个 W2 矩阵使用各自实现的 K=768 内核。每个矩阵最后一次调用不足 288 行时用零权重补齐，只复制有效输出，因此多计算了 1,408 个填充行，使实际整数乘加项为 15,593,472，比有效工作量多 2.67%。

3.8.3 完整计算链的验证结果

固定模型已分别使用标准 RVV 内核（下称 FR）和 VDot 内核（下称 TA）在香山仿真模型上完成完整前向计算验证。两路均完成 3,691,424 字节的全状态比较，并核对输出保护区、调用状态及浮点异常标志。表3-6是这次正确性实验中两条路径各一次的计数窗口；TA 独立重放得到相同的完整验证结果和窗口计数。

表 3-6 完整 forward 正确性实验中的单窗口目标机观测

实现	周期数	退休指令数
FR：标准 RVV	20,258,039	7,502,872
TA：自定义 VDot	19,774,245	10,122,100

从表中可以看到，在完整前向这一层面，TA 比 FR 少用约 2.4% 的周期，退休指令却多约 35%。收益远小于微基准，可以从两方面理解：一是完整前向中还有尺度换算、访存、注意力和归一化等非点积运算；二是量化 GEMV 按每 32 项一组计算，八项 vdot 每条只处理 8 项，一组需要 4 条 vdot 及相应的累加，指令条数较多。第二阶段的十六项扩展每条处理 16 项，可以减少这部分指令。表中两次均为单窗口观测，也说明退休指令数不能代替周期来比较性能。

3.9 综合面积、时序与设计成本

设计成本使用 Nangate45 Typical 工艺库（1.10 V、25 °C）评估，工具为 Yosys 0.9、ABC 1.01 和 OpenSTA 2.0.17。评估对象有两个：独立的 FU，以及包含 VDot、DIV/SQRT 和共享控制的完整父级。两者都以 4 ns 为目标综合，再对得到的网表做布局前 setup 检查。综合使用受测版本的 RTL，其中两槽流水、32 位结果保存和 DIV 取消修复均已启用，调试追踪关闭；版本标识见表3-14。SP1 方案的结果见第3.9.1节。

约束周期取 4 ns，是因为 Nangate45 是 45 nm 开源教学库，门延迟远大于香山实际采用的工艺。在这一库下，父级中原有的除法等逻辑在 1 GHz 附近无法闭合；第3.9.2节的数据也显示，即使换用 7 nm 库，父级在 1 ns 下仍有负裕量。选用父级能够闭合的 4 ns 作为统一约束，各结构的面积和路径余量就都在可实现的条件下比较。这个周期只用于成本比较，不代表香山的工作频率。

综合时 ABC 的延迟目标为 4000 ps，输入由 BUF_X1 驱动，输出负载为 5 fF。STA 使用 4 ns 时钟周期、5 fF 负载和 `5K_hvratio_1_1` 线负载模型；输入、输出延迟各 0.1 ns，时钟不确定度 0.05 ns，输入和时钟转换时间各 0.05 ns；关闭时钟重汇聚悲观消除（CRPR），父级的测试使能 `testEnable` 置 0。结果见表3-7。

父级包含 FU 和三个时钟门控单元，两项面积不能相加。表中面积是综合后的标准单元面积

表 3-7 4 ns 综合面积与 4 ns / 5 fF STA 结果

指标	独立 VDot FU	完整 VFEX4 父级
标准单元面积（μm^2）	17112.578	284562.278
寄存器至寄存器到达时间（ns）	1.200609	3.483995
该类路径 setup 裕量（ns）	+2.722349	+0.436214
全局最差 setup 裕量（输入到寄存器，ns）	+2.601416	+0.422254
各类路径 setup 裕量	5 类全部为正	9 类全部为正

之和，不是布局布线后的芯片面积。在 4 ns 约束下，FU 和父级的所有路径都满足 setup 要求，两者最差的都是输入到寄存器路径，裕量分别为 +2.60 ns 和 +0.42 ns。父级的最差路径不在 VDot 上，与 VDot 相关的路径裕量都在 1 ns 以上。

图3-10给出面积占比和全部 14 类路径的 setup 裕量。i2o、i2r、r2o、r2r 分别表示输入到输出、输入到寄存器、寄存器到输出和寄存器到寄存器路径，带 vdot 前缀的是父级中与 VDot 相关的路径。FU 面积约占父级的 6.0%；14 类路径的裕量全部为正，其中父级中 VDot 结果送往输出端口的路径裕量为 +1.07 ns，时钟门控使能路径为 +1.99 ns。

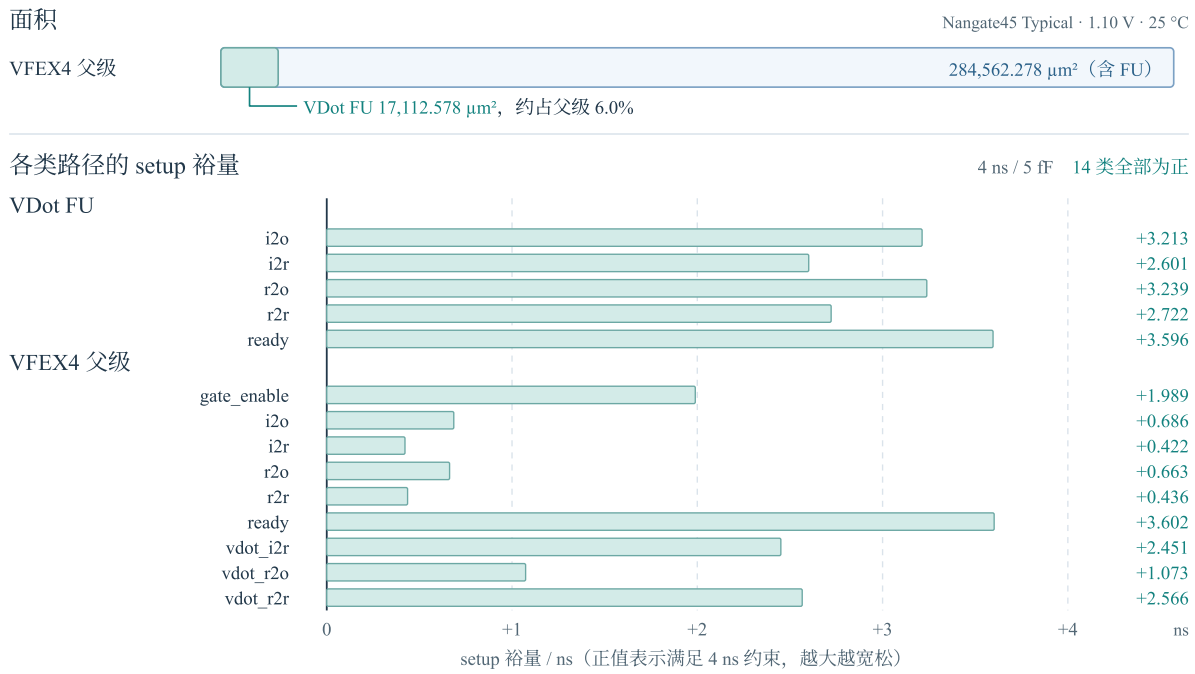


图 3-10 Nangate45 模块面积与 4 ns / 5 fF STA 裕量

以上是局部模块的布局前 setup 分析，尚未覆盖 hold、电气约束、多工艺角、布线后时序和功耗，也没有对综合网表做形式等价或门级仿真，因此不代表整核的物理频率。约束、原始路径报告和输入版本信息保存在 evidence/nangate45-4ns-20261108/。

3.9.1 SP1 一拍运算方案的对比

稳定版的点积运算分两拍完成，结果经两个槽位弹性输出。为缩短单条点积的延迟，SP1 方案把乘法和归约合并到一拍，两槽结构以及反压、取消和与 DIV/SQRT 共用执行单元的协议保持

不变。两版的提交标识见表3-14。SP1 已通过项目验收清单的全部 20 项检查，清单涵盖微基准性能、C8 调用的功能与性能、控制协议和 DIV/SQRT 共用测试。

表3-8的周期来自此前的配对测量：两版运行同一程序镜像，d0、d1、d2 三组数据的结果相同，C8 软件也完全相同。加速比等于同一负载下 RVV 平均周期除以 VDot 平均周期。

表 3-8　稳定版与 SP1 的周期及相对 RVV 加速比

负载	稳定版周期	SP1 周期	稳定版加速比	SP1 加速比
依赖链（256 次）	1857	1601.25	2.9297×	3.3977×
单次点积	71	70	1.2007×	1.2179×
四路独立批量（128 次）	221	221	3.0679×	3.0679×
加载、计算、存储流（64 次）	1127.5	1126.25	1.0049×	1.0060×
C8 配置复用调用	273	272	1.0952×	1.0993×

注：周期为测量窗口内的平均 DUT 周期。五行对应的 RVV 周期依次为 5440.5、85.25、678、1133、299，两版相同。

SP1 的收益集中在串行依赖链：VDot 周期减少 13.8%，加速比由 2.93 倍提高到 3.40 倍。批量场景中的点积彼此独立，两槽结构本来就能连续发起，缩短单条延迟不再提高吞吐，因此周期不变；C8 配置复用调用只从 273 降到 272 周期，减少 0.37%。

成本比较沿用第3.9节的工具、工艺库和约束：两版都以 4 ns 为目标综合，再做 4 ns、5 fF 的 STA，分析脚本完全相同，SP1 同样关闭调试追踪。只看父级最差裕量会掩盖 VDot 路径本身的变化，所以表3-9同时列出全局最差路径和与 VDot 直接相关的路径。

表 3-9　两版的 4 ns 综合面积与 4 ns / 5 fF 时序

指标	稳定版	SP1
独立 FU 面积（μm^2）	17112.578	18156.628
完整 VFEX4 父级面积（μm^2）	284562.278	285182.058
独立 FU 全局最差 setup 裕量（ns）	+2.601416	+1.951567
父级全局最差 setup 裕量（ns）	+0.422254	+0.493821
父级 VDot 输入至寄存器裕量（ns）	+2.450936	+1.742610
父级 ready 返回到达时间（ns）	0.247821	0.303064
各类路径及全局 setup 检查	全部通过	全部通过

注：父级包含 FU，两项面积不能相加。裕量越大越宽松，到达时间越小越快。到达时间包含约束设定的输入到达和时钟条件，不等于纯组合逻辑延迟。

与稳定版相比，SP1 的 FU 面积增加 6.1%，父级面积增加 0.22%。把运算压进一拍后，VDot 从输入到寄存器的单拍组合路径变长：FU 最差裕量减少 0.65 ns，父级中 VDot 输入至寄存器路径的裕量减少 0.71 ns，ready 返回路径的到达时间增加 0.055 ns。父级全局最差路径不在 VDot 上，其裕量略增 0.072 ns。两版在 4 ns 下都满足 setup 要求，但 SP1 的 VDot 路径余量明显缩小。

逐路径数据、约束一致性检查和周期数据的出处见 evidence/sp1-4ns-comparison-20261108/。

3.9.2 不同工艺库下的参考综合

为了解同一设计换用更先进的工艺库后面积与时序如何变化，本节引用团队于 2026 年 10 月 28 日完成的双工艺库评估。该实验把同一份 RTL 分别用 Nangate45 和 ASAP7 综合，对象同样是独立 VDot FU 和完整 VFEX4 父级（含 VDot、整数 DIV、浮点 DIV/SQRT 和三个时钟门控单元）。所用 RTL 同样启用两槽流水和 32 位结果保存，但与受测版本不是同一提交，版本标识见表3-14。

两库使用相同的工具和约束：ABC 目标 1 ns，零互连线负载，输入、输出延迟各 0.1 ns，时钟不确定度 0.05 ns，输入和时钟转换时间各 0.05 ns，主档为 1 ns 时钟、5 fF 负载。Nangate45 为 Typical、1.10 V、25 °C；ASAP7 为 7.5-track、RVT、TT、NLDM 库，0.70 V、25 °C，即两库各自的标称条件。由于源码版本、综合目标、STA 周期和线负载模型都与前文的 4 ns 结果不同，本节数据只在两库之间对比，不与表3-7混算。

表3-10列出两库的标准单元面积。面积取自 Liberty 中的单元面积，并已与 1 倍尺度 LEF 中的单元宽高逐项核对，不是按 45 nm 与 7 nm 的比例推算。换用 ASAP7 后，FU 和父级面积分别减少 92.7% 和 93.2%。这一差异来自工艺库本身，不属于 RTL 优化收益。

表 3-10 同一 RTL 配对综合的标准单元面积（μm^2）

统计对象	Nangate45	ASAP7 RVT/TT	面积减少
独立 VDot FU	17266.858	1263.05082	92.6851%
完整 VFEX4 父级（含 FU）	284935.742	19311.00588	93.2227%

注：数据来自 1 ns 目标综合，减少比例以同一行的 Nangate45 面积为基准。面积为布局前标准单元面积之和；父级包含 FU，两项不相加。

面积大幅下降，时序却没有全面改善，见表3-11。ASAP7 下独立 FU 在 1 ns 时有正裕量，但完整父级的最差裕量仍为 −0.65 ns，要放宽到 2 ns 才全部满足；其中与 VDot 相关的输入至寄存器路径只差约 0.07 ns。父级 ready 返回路径的到达时间还从 0.244 ns 增加到 0.268 ns。两库的 STA 都关闭了 CRPR，因为没有实际时钟树时，这项补偿缺少依据。

表 3-11 双工艺库配对评估的布局前时序（ns）

指标与约束	Nangate45	ASAP7 RVT/TT
独立 FU 最差 setup 裕量（1 ns / 5 fF）	−0.181771	+0.118721
父级最差 setup 裕量（1 ns / 5 fF）	−2.006459	−0.651433
父级最差 setup 裕量（2 ns / 5 fF）	−1.006459	+0.348567
父级 VDot 输入至寄存器裕量（1 ns / 5 fF）	−0.695828	−0.069247
父级 ready 返回到达时间（1 ns / 5 fF）	0.243529	0.267816

注：网表均按 1 ns 目标综合，STA 周期见各行。所用 RTL 为两拍运算的两槽实现。

ASAP7 是预测性的 7 nm FinFET 学术工艺库。本组数据同样只是布局前综合结果，没有布局布线、时钟树综合、hold 与电气检查、多工艺角和功耗分析，它说明的是工艺库对面积和关键路径的影响，不代表整核能达到的频率，也不改变任何周期结果。详细数值和条件见独立 FU 双库评估与完整父级双库评估，原始结果摘要保存在 evidence/asap7-reference-20261108/。

3.10 设计取舍与应用方向

综合性能、面积和时序，最终提交采用稳定版硬件，软件使用首段初始化版本和 C8；SP1 作为已通过验收的优化方案保留。SP1 的收益集中在依赖链（周期减少 13.8%），批量吞吐不变，C8 配置复用调用只快 0.37%；代价是 FU 面积增加 6.1%，VDot 输入至寄存器路径的裕量减少约 0.65 至 0.71 ns，ready 返回也变慢。在 4 ns 下两版都还有余量，但这条路径在更高频率下会更早变紧：双工艺库数据显示，换用 7 nm 库并收紧到 1 ns 时，即使是两拍实现，这条路径也只差约 0.07 ns，而一拍方案把乘法和归约放进同一拍，路径更长。权衡有限的应用收益和这项时序风险，最终选择稳定版。

两槽流水提高了独立点积的发起速率；结果只保存 32 位，利用的是结果高 96 位恒为零；DIV 取消修复使共享资源能在取消后及时释放。软件侧则通过首段初始化、C8 配置复用及更适合量化组的 RVV 组织减少实际调用开销。这些优化作用在不同环节，通过配对实验可以把各自的收益和代价分开衡量。

应用方面，固定 GEMV 已经给出普通 C 调用的周期结果，完整前向计算验证了量化内核能正确接入整个模型。完整前向的分析也指出了下一步：十六项点积每条处理 16 项，可以减少量化 GEMV 的指令条数，目前已完成功能验证，下一步是接入完整模型并与较快的 RVV 实现比较。布局后时序、整核频率和功耗需要在后续物理实现中评估。

图3-11汇总受测版本在各层级已有和待补的评估。功能正确性和周期在四个层级都有记录；面积和布局前 STA 覆盖局部 FU 和 VFEX4 父级，在 4 ns / 5 fF 约束下 setup 全部满足；功耗和整核层级的物理指标仍待评估。FU 的握手拍与整核、C8 的 DUT 周期分别计量。

最终交付组合：两槽硬件 + 首段初始化 + C8　● 已有记录　○ 待评估　— 随硬件评估

评估对象	逻辑与执行		物理实现		
	功能正确性	周期测量	单元面积	布局前 STA	功耗
局部 VDot FU 算术与握手	●	●	●	●	○
VFEX4 父级 含 FU 与 DIV	●	●	●	●	○
整核程序 差分／微基准	●	●	○	○	○
固定 GEMV / C8 完整调用	●	●	—	—	—

周期单位	FU 按握手拍计；整核与 C8 按 DUT 周期计
功能验证	全核 117 个用例共运行 124 次；C8 最终镜像与测试版一致
时序结果	4 ns／5 fF 约束下，14 类路径 setup 全部满足
后续评估	整核最高频率、布线后时序与功耗
复现方式	源码、镜像、输入和原始记录均已归档

图 3-11　受测版本的验证与评估范围

3.11 补充：三种标准 RVV 实现的完整前向观测

本节记录一组只涉及标准 RVV 实现的补充实验，说明软件组织方式对完整前向计算的影响。它不包含 VDot 指令，因此不用于评价 VDot 的收益。

在相同模型与共同程序上，进一步接入了两种标准 RVV 软件组织：GS32 一次处理完整的 32 项量化组；paired16 把前后半组对应的乘积在 16 个 INT16 通道中相加，再扩宽归约。paired16 在入口检查激活值，遇到 −128 时执行完整 FR 路径，保持完整的 signed-int8 数值域。入口检查和回退的开销也计入调用。

三路共同程序已完成原 NEMU 参考验证，并于 2026 年 11 月 6 日完成三路香山目标运行。三种方法共用 ELF（SHA256 前缀 `be6c3ead424d`），目标镜像仅四字节方法选择字不同；W2 继续使用 FR，实现差异集中在输入维度为 288 的矩阵路径。三个目标运行均通过原 DiffTest、3,691,424 字节完整状态比较、保护区与调用状态检查，浮点异常标志也与参考结果完全一致。计时用的四条 CSR 读取指令均核对了采样和退休时刻；三条标准 RVV 路径都没有执行自定义点积指令，符合预期。

表3-12列出同一构建的三个完整 forward 目标窗口；目标退休指令数与各自已有 NEMU 参考记录相同。

表 3-12 共同 ELF 的完整 forward 单窗口目标观测

实现	目标窗口周期数	目标窗口退休指令数	相对 FR 周期减少
FR	22,244,288	7,502,886	—
GS32（G）	19,996,570	4,828,788	10.1047%
paired16（P）	17,382,632	5,175,371	21.8558%

在本次固定窗口中，paired16 比 GS32 少用 13.1% 的周期，同时多退休 346,583 条指令，即增加 7.2%。因此不能只看指令数来判断两种实现谁更快。

本组使用固定合成模型，计时覆盖共同 prepare 之后、position=0、token=1 的一次完整 forward，不包含模型生成或载入、离线 packing、程序启动及此前的初始化和 prepare。参考验证另外做了 paired16 独立重放和末项输出扰动负控；目标机上只运行了三次正常观测，没有再做重放或负控。

表中百分比来自单次观测。这组结果说明，软件的数据组织方式对完整前向的周期影响很大。

本组 FR 与第3.8节的 FR 使用同一套仿真环境，退休指令数只差 14 条，周期却多约 9.8%。两者属于不同的软件构建：本组重新编译了包含三种 RVV 路径的共同程序，方法选择也放在计时窗口内，程序布局随之改变。周期差异的具体来源尚未做微架构层面的分析，因此两组数据只在各自组内比较，不跨组计算加速比。

3.12 补充：C8 软件包的接入与镜像核对

本节给出 C8 软件包的接口约定、调用方式和最终镜像核对过程，供接入和复查使用。

3.12.1 固定平台与数据布局

C8 软件包面向本项目已验证的 RV64、lp64d、VLEN128 裸机环境，调用前启用向量状态。VDot 路径使用项目的自定义点积编码；RVV 路径使用标准向量指令。输入、输出各按 16 字节对齐并独立存放，输入在调用期间保持有效。接口约定列于表3-13。

表 3-13 C8 接口约定

项目	接口约定
输入	行优先 $A[8][32]$ 后接 $x[32]$，共 288 字节
输出	8 个 16 字节槽，共 128 字节
单槽	低 32 位 INT32 结果，高 96 位零
返回向量配置	`vl=8, vtype=0, vstart=0, vcsr=7`

函数沿用普通 RV64 C ABI 保存栈和被调用者保存寄存器。函数内部可以使用全部向量寄存器，返回配置固定为 `e8/m1/tu/mu`。结果槽宽度对应 VLEN128 下的整寄存器存储，因此调用方按照 16 字节步长读取各行结果。

3.12.2 编译期选择与调用

头文件 `selected_gemv_c8.h` 提供两个入口。`xs_c8_gemv_8x32` 默认连接配置复用 VDot；将 `XS_C8_GEMV_FORCE_RVV` 设为字面量 1 后，该名称连接 RVV 实现。`xs_c8_gemv_8x32_rvv` 始终连接 RVV。开关为 0 或 1，选择过程在预处理阶段完成。切换版本或开关后，重新编译相关翻译单元并链接匹配的静态库。

```
#include <stdint.h>
#include "selected_gemv_c8.h"

struct c8_slot {
    int32_t value;
    uint32_t upper_zero[3];
};
static _Alignas(16) int8_t packed_ax[288];
static _Alignas(16) struct c8_slot result[8];

/* Fill packed_ax with A[8][32] followed by x[32]. */
void run_c8(void) {
    xs_c8_gemv_8x32(packed_ax, result);
}
```

需要标准 RVV 对照时，可在同一调用位置使用显式 RVV 入口。保留首段初始化版本的应用继续使用其对应头文件和库；C8 以独立名称加入软件包，便于按具体形状选择和回退。[3]

3.12.3 从函数对象到可执行镜像

软件包保留一个完整 `gemv.o`，由 `libxs_gemv_c8.a` 提供 16 个公共函数。固定 GNU 参数为`-march=rv64gcv -mabi=lp64d -mno-relax`。验证分别核对编译期别名、实际

[3]头文件及对齐示例见C8 编译期入口与同目录 `example.c`。接口细节见C8 接口规范。

调用重定位、库成员抽取和最终装载内容，使每个公开入口都能追溯到被测试的函数。

SDK 接入的三方比较覆盖 BIN、PT_LOAD、BSS、函数地址与机器码、调用表及输入数据。三份 BIN 均为 31,440 字节且内容一致；去除调试节后的 ELF 也逐字节一致。调试信息因源目录和头文件路径不同而有差异，这部分单独保留，不影响运行内容的比对。图3-12给出完整的核对流程：先固定全部输入文件，再从 gemv.o 抽取库成员并链接，比较三方最终 BIN、装载段、调用记录和输入；完整 ELF 的差异仅在调试节。右侧的 6 项结构与输入负控、1 项错误空库和 1 项调试正控均得到预期结果。因此，原测试程序的运行结果对封装后的软件包同样有效。[4]

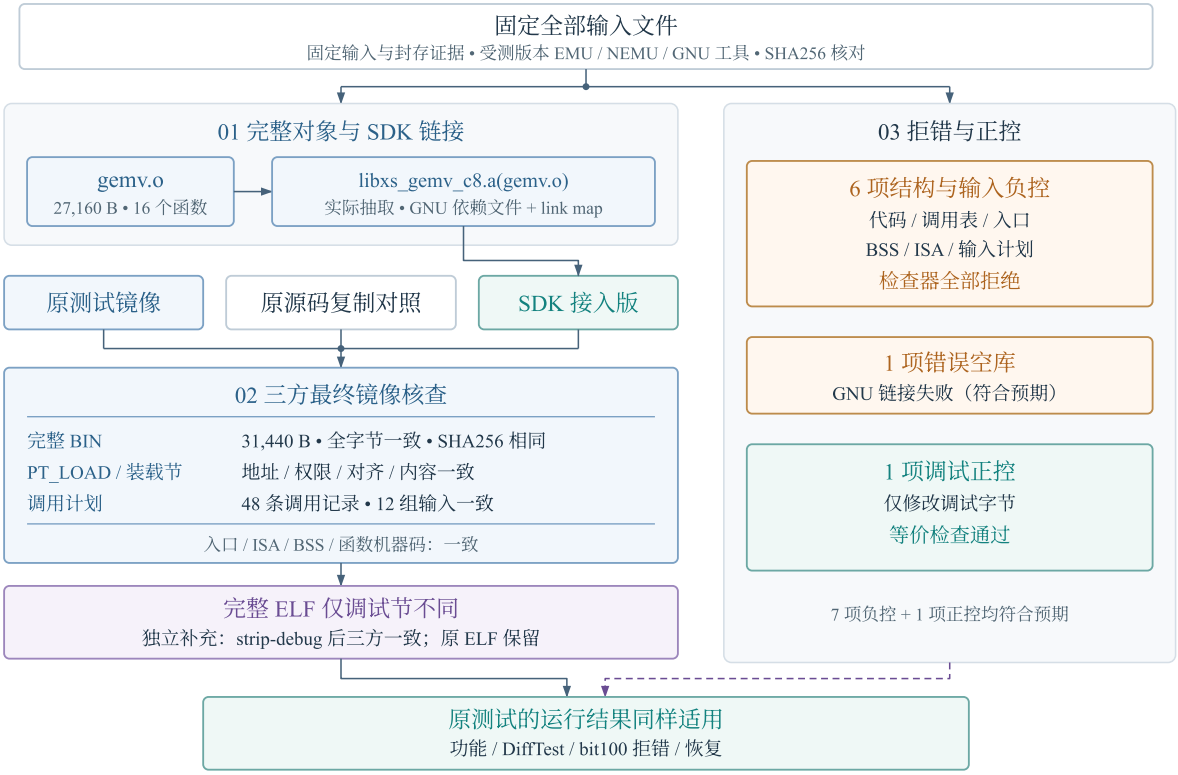


图 3-12　C8 接口、静态库与最终镜像的核对流程

复现方法见软件包的 README、固定输入清单及配套核验工具。

3.13　评估对象与证据索引

报告按实际受测对象组织结果。四类微基准、普通 C 调用和完整 forward 各自对应固定的代码、程序与输入；可沿表3-14定位产生每组结果的实验。

表 3-14　评估对象与版本标识

评估对象	版本标识	对应结果
硬件微基准	受测版本：硬件提交 0f12df6753 及对应 EMU	四类微基准的正式窗口和同镜像比较
干净环境复现	干净环境复现构建（归档名 clean-02）及其参考模型	干净环境构建、差分和工程验收

[4]完整比较记录、链接映射及独立 GNU 复核见C8 最终镜像等价结果。

表 3-14 评估对象与版本标识（续）

评估对象	版本标识	对应结果
C8 普通 C 入口	最终 BIN 的 SHA256 前缀 `56805506eb98`	固定 GEMV、调用状态与最终镜像等价
Nangate45 综合与 STA	受测版本：硬件提交 `0f12df6753`（源码与发布提交 `a0f13456e0` 相同）	4 ns 综合面积与 4 ns / 5 fF setup 结果，见第3.9节
SP1 对比与选型	SP1：提交 `ffb52e4504`；稳定版：提交 `a0f13456e0`	两版周期、4 ns 面积与 STA 对比，最终保留稳定版，见第3.9.1节
双工艺库参考综合	源码提交 `e1815c0732`	Nangate45 与 ASAP7 的面积和布局前 setup 对比，见第3.9.2节
完整前向（FR 与 TA）	同一套仿真环境（硬件提交 `34bae6d4`）与两种调用程序	首个 token 完整执行、全状态比较及目标机单窗口观测
三种 RVV 实现的完整前向	同一套仿真环境、共同 ELF `be6c3ead424d` 及固定模型	完整状态比较、参考重放与负控，以及三个正常目标窗口的周期和退休指令观测

四类微基准可由微基准原始窗口 CSV与测量方法与成本汇总复查；软件包的使用与版本选择见C 入口与版本间比较。

完整前向计算的FR/TA 目标机记录和FR/GS32/paired16 共同参考结果分别对应各自的实验；本次三路 RVV 目标观察见三路目标结果及证据入口。各实验目录同时保存构建信息、输入清单和原始运行记录。

3.13.1 复现与图像来源

本报告的数据截至 2026 年 11 月 8 日，产生各组数据的构建见表3-14。原始日志、失败记录及构建清单随各实验目录保存在团队仓库中，复现者可从各目录的 RESULT、测量规范和输入清单找到对应的原始记录。第3.9节引用的综合约束、路径报告和输入版本信息另存于报告工程的`evidence/`目录。

技术图依据源码、测试记录及测量数据绘制，终端与 GTKWave 截图保留实际观察内容。